Informe de refactorización de la arquitectura ROS 2 del Bípedo

Autores
Hean Felipe Gómez Gonzalias — 2230669
Santiago Sarria Arcila — 2215465

---

1. Resumen

Se revisó el repositorio `BABAKAR05/Bipedo_Mu-eco` para inventariar paquetes, nodos, tópicos, launches y dependencias. A partir de esa revisión se hicieron cambios de estructura, de código y de documentación. También se cambió el firmware de la ESP32: los servos ya no se controlan con un PCA9685, sino con PWM directo desde los GPIO.

- Los launches se redujeron de 10 a 5: nueve duplicados se reemplazaron por cuatro launches parametrizados. `sim.launch.py` no cambió.
- Se eliminaron 36 archivos, incluidos un paquete ROS 2 completo (`robot_serial_bridge`) y la carpeta de firmware `control_pata_sin_ros` .
- Los cinco nodos de cinemática inversa se unificaron en `ik_node` , sin cambiar sus tópicos.
- La matemática de cinemática (DH, cinemática directa y límites) se centralizó en `kinem_common.py` .
- El firmware controla seis servos con PWM directo en los GPIO 18, 19, 23, 25, 26 y 27, con grados lógicos, cero en 1500 µs y movimiento suave.
- La documentación (`ARCHITECTURE.md` , README del firmware y el documento de entorno Ubuntu) se actualizó para coincidir con el código.

---

2. Alcance

Los cambios se hicieron en la rama `refactor/auditoria-arquitectura`, sin tocar `main`. El trabajo tuvo dos partes: primero una revisión de solo lectura para inventariar el proyecto, y después los cambios aprobados por el equipo.

---

3. Arquitectura antes y después

Antes

- 8 paquetes ROS 2, incluido `robot_serial_bridge`.
- Dos launches por pierna casi idénticos, y siete launches de descripción y Gazebo también duplicados.
- Cinco nodos de cinemática inversa, cada uno en su propio proceso.
- La matemática DH copiada en unas seis partes, y los límites articulares en cuatro lugares.
- Dos firmwares para los mismos servos: `esp32_servos` por micro-ROS y `control_pata_sin_ros` por puerto serie.
- Documentación con valores y nodos que no existían en el código.

Después

- 7 paquetes ROS 2.
- Un launch de bringup (`bringup.launch.py leg:=right|left`) y tres launches de modelo (`description` , `display` , `gazebo`) con `model:=right|left|bipedo`.
- Un nodo `ik_node` con multihilo para los cinco métodos de cinemática inversa.
- Un módulo `kinem_common.py` como única fuente de la matemática común.
- Un solo firmware, `firmware/esp32_servos`, con PWM directo a seis servos.
- Documentación alineada con el código.

---

4. Cambios realizados

4.1 Launches consolidados

Cambio	Archivos	Justificación
Un launch de bringup para ambas piernas	`bringup.launch.py` reemplaza `bringup_der.launch.py` y `bringup_izq.launch.py`	Eran casi idénticos. Solo cambiaban nombres de joints y `leg_side`. Ahora `leg` decide la pierna.
Launches de descripción parametrizados	`description`, `display`, `gazebo` reemplazan siete archivos	Los pares der/izq solo cambiaban el nombre del XACRO. `model:=bipedo` cubre el muñeco completo.

```
ros2 launch robot_bringup bringup.launch.py leg:=right
ros2 launch robot_description display.launch.py model:=bipedo
```

4.2 Código sin uso eliminado

Archivo	Motivo
`kinem_leg_gen.py`	Nadie lo importaba. Sus longitudes eran distintas a las actuales. La documentación decía que lo usaba `kinematics_node`, y no era así.
`kinem_leg_rtb.py`	Nadie lo importaba. Dependía de `roboticstoolbox`, que no está declarado en el paquete.
`forward_kinematics.py` y su test	Duplicaba la cinemática directa con la cadena obsoleta. El test validaba esa cadena, así que se eliminó.
`balance.py` y `sensor_processing.py`	Stubs sin importaciones, para trabajo de IMU que no está en el alcance actual.
`prueba.txt`, `tests/Hola_mundo.py`, `joint_names.yaml`	Sin función en el proyecto. El último no lo cargaba ningún launch ni nodo.

Se conservaron `pid_controllers.py` y `motor_control.py`, porque se usan en el modo PID de `control_node` (`verify_mode:=false`).

4.3 Matemática común centralizada

Se creó `robot_kinematics/kinem_common.py` con las longitudes `L1..L5`, los límites articulares, `dh_matrix`, `fk_T` y `joint_limit_warnings`. Los módulos

`kinem_invers_leg_*.py` y `cinematica_directa_der_izq.py` importan ahora estas funciones. Las copias eran matemáticamente idénticas. La única diferencia era el formato del aviso de límites, y se adoptó la versión con el valor del ángulo y el símbolo de grado correcto.

4.4 Cinemática inversa en un solo nodo

`ik_node.py` atiende los cinco métodos en un solo proceso:

Método	Objetivo	Resultado	Tipo
Algebraico	`/robot/ik_target`	`/robot/ik_result`	Point
Newton-Raphson	`/robot/ik_newton_target`	`/robot/ik_newton_result`	IKJacobTarget
Desacople	`/robot/ik_des_target`	`/robot/ik_des_result`	Point
Geométrico	`/robot/ik_geom_target`	`/robot/ik_geom_result`	Point
Gradiente descendente	`/robot/ik_grad_target`	`/robot/ik_grad_result`	IKJacobTarget

- Cada método tiene su grupo de callbacks y el ejecutor es multihilo. Así, el gradiente descendente (hasta 10 000 iteraciones) no bloquea a los demás.
- Los tópicos y tipos de mensaje no cambiaron, así que `teleop_node` no se modificó.
- Se eliminaron `ik_newton_node`, `ik_des_node`, `ik_geom_node` e `ik_grad_node`, y sus entradas en `setup.py`.

4.5 Firmware: de PCA9685 a PWM directo

El PCA9685 se quitó y los seis servos se conectaron directamente a los GPIO de la ESP32. El firmware también añadió el cero lógico, las escalas, el sentido por servo y el movimiento suave.

Articulación	GPIO	Posición en el mensaje	Escala	Sentido	Límites lógicos (°)
IZQ Hip Roll	18	data[0]	270°	−	−15 a 90
IZQ Hip Pitch	23	data[1]	180°	−	−90 a 90
IZQ Knee	19	data[2]	180°	+	−90 a 90
DER Hip Roll	26	data[3]	270°	+	−15 a 90

Articulación	GPIO	Posición en el mensaje	Escala	Sentido	Límites lógicos (°)
DER Hip Pitch	27	data[4]	180°	+	−90 a 90
DER Knee	25	data[5]	180°	−	−90 a 90

- **Cero lógico:** 0° equivale a 1500 µs, la posición física de reposo. El firmware convierte grados a microsegundos con `us = 1500 + dir × grados × µs/grado`.
- **Movimiento:** actualización cada 20 ms (50 Hz), velocidad máxima de 60 °/s y aceleración máxima de 120 °/s². Cada servo frena antes del objetivo y no lo sobrepasa.
- **Protocolo `/servo_commands`:** seis valores en grados lógicos. Un `NaN` significa "no cambiar ese servo". Un mensaje con menos de seis valores se ignora.
- **Reposo al arrancar:** los seis servos van a 0° lógico, de uno en uno.
- **Pérdida de comandos:** los servos mantienen la última posición.

> **Nota:** las escalas, los límites, el reposo y la velocidad son valores nominales. Se calibrarán con el robot.

4.6 Teleop

- Envía en `/servo_commands` los tres valores de su pierna en grados lógicos, y `NaN` para la otra. Por eso `servo_offset_deg` quedó en cero.
- El rango del slider de cadera roll pasó a −15° a 90°, igual que el firmware y `control_node`.

4.7 Otros cambios

- `JointState.msg`: el comentario mencionaba `Ankle_Joint`, que ya no existe. Ahora usa los nombres reales de los joints.
- `control_node.py`: el comentario apuntaba a un launch eliminado, y se actualizó.
- El firmware dejó de usar `Adafruit PWM Servo Driver` y `Adafruit BusIO`. Ahora usa `madhephaestus/ESP32Servo`.

4.8 Documentación

- `ros2_ws/src/ARCHITECTURE.md` describe el código real: paquetes, tópicos, cuatro lugares de límites, sentido de la rodilla y pasos de prueba.

- `firmware/esp32_servos/README.md` explica la convención de grados lógicos, la tabla de servos, el protocolo y los pasos de carga desde Ubuntu.
- `docs/cambios_ubuntu_ignorar.md` resume los cambios de firmware y teleop, y el entorno de la máquina de pruebas.
- `README.md` refleja la estructura real del repositorio y añade `robot_bringup`.

---

5. Verificación

Verificación	Resultado
Compilación de sintaxis de todos los archivos `.py` del repositorio	Sin errores
Regresión numérica de cinemática directa e inversa, comparando la versión anterior con la nueva	Resultados idénticos en ambas piernas, en los cinco métodos de cinemática inversa y en `verificar_con_mth`
Nodo `ik_node` con stubs de `rclpy` y de los mensajes	Cada tópico publica el resultado de su solver
Búsqueda de referencias a los archivos eliminados	Sin referencias restantes

> Estas pruebas se hicieron sin ROS 2 ni hardware. La ejecución del workspace con ROS 2 y las pruebas con el robot físico no forman parte de este informe.

---

6. Anexo: archivos

Eliminados (36): 9 launches duplicados, 10 archivos de código, el paquete `robot_serial_bridge/` (6 archivos), la carpeta `firmware/control_pata_sin_ros/` (8 archivos) y tres archivos sin función (`joint_names.yaml` , `prueba.txt` , `tests/Hola_mundo.py`).

Creados (5): `bringup.launch.py` , `description.launch.py` , `display.launch.py` , `gazebo.launch.py` y `kinem_common.py` .

Modificados (principales): `ik_node.py`, `cinematica_directa_der_izq.py`, seis módulos `kinem_invers_leg_*.py`, `setup.py` de `robot_kinematics`, `control_node.py`, `teleop_node.py`, `JointState.msg`, los archivos del firmware `esp32_servos` (`main.cpp`, `servo_params.h`, `helpers.h`, `pins.h`, `platformio.ini`, README), `ARCHITECTURE.md`, `README.md` y los documentos de `docs/`.